

Corso di Laurea in Ingegneria e Scienze Informatiche

Analisi dell'impiego di SOG e COG in algoritmi di clustering per reti marittime

Tesi di laurea in:
PROGRAMMAZIONE AD OGGETTI

Relatore

Prof. Danilo Pianini

Candidato

Lorenzo Antonioli

Correlatore

Dott. Martina Baiardi

Sommario

Max 2000 characters, strict.

Optional. Max a few lines.

Indice

Sommario	iii
1 Introduzione	1
2 Background	3
2.1 Comunicazioni marittime e AIS	3
2.1.1 Funzionamento del sistema AIS	3
2.1.2 Struttura e codifica del messaggio AIS	4
2.1.3 Aspetti normativi e applicazioni	6
2.1.4 Limiti e problemi aperti	6
2.2 Aggregate Programming, SCR e Kollektive	7
2.2.1 Programmazione Aggregata (AP)	7
2.2.2 Field Calculus (FC) e Building Blocks	8
2.2.3 Self-organising Coordination Regions (SCR)	10
2.2.4 Kollektive	11
2.3 Simulazione in ambito marittimo	14
2.3.1 Modeling & Simulation (M&S)	14
2.3.2 Alchemist	14
2.4 Algoritmo precedente	18
3 Analisi	21
3.1 Limiti dell'algoritmo esistente	21
3.2 Obiettivi	21
3.3 Requisiti	21
4 Design e implementazione	23
4.1 Strategia di integrazione nel criterio di clustering	23
4.2 Estrazione e integrazione delle feature AIS aggiuntive	23
4.3 Modifiche all'algoritmo	23

5	Valutazioni	25
5.1	Risultati dell'esperimento precedente	25
5.2	Risultati del nuovo algoritmo	25
5.3	Confronto e discussione	25
6	Conclusioni e sviluppi futuri	27
6.1	Limitazioni dello studio	27
		33
	Bibliografia	33

Capitolo 1

Introduzione

Capitolo 2

Background

2.1 Comunicazioni marittime e AIS

Le comunicazioni marittime si basano su una combinazione di sistemi radio Very High Frequency (VHF) e satellitari (Satellite Communications (SatCom)) per garantire la sicurezza della navigazione e l'efficienza delle operazioni [BPAFT25]. Mentre i sistemi satellitari assicurano una copertura globale, (comportando latenze e costi maggiori), i sistemi VHF supportano lo scambio di dati a corto raggio (in linea di vista o Line of Sight, tipicamente a 40 km a seconda del meteo) per i collegamenti sia tra imbarcazioni (Ship to Vessel (S2V)) sia verso le stazioni costiere (Ship to Shore (S2S)) [BPAFT25]. Una delle tecnologie fondamentali basate sui sistemi radio è l'Automatic Identification System (AIS). L'AIS è un sistema di identificazione automatica che permette alle navi di trasmettere e ricevere in modo continuo informazioni sulla propria posizione, rotta, velocità e stato di navigazione, utilizzando la banda radio VHF dedicata [Int26].

2.1.1 Funzionamento del sistema AIS

L'AIS è nato come strumento per aumentare la sicurezza in mare, affiancando i radar per ridurre il rischio di collisioni e migliorare la “consapevolezza situazionale” dei comandanti [Int26, Uni15]. Ogni transponder AIS invia periodicamente messaggi brevi (tipicamente ogni 2–6 secondi) che includono sia dati statici (identificativo International Maritime Organization (IMO), Maritime Mobile Service

Identity (MMSI), tipo di nave, dimensioni) sia dati dinamici (posizione Global Positioning System (GPS), rotta, velocità, stato di navigazione), oltre a dati di viaggio come la destinazione prevista [Int26]. Questi messaggi possono essere letti da altri transponder a bordo (modalità S2V) e da stazioni costiere (S2S), permettendo il monitoraggio del traffico da parte delle autorità (Vessel Traffic Service (VTS), Guardia Costiera) [Int26, Uni15] e anche da servizi commerciali (es. portali come MarineTraffic¹ o VesselFinder²). La portata tipica è di alcune decine di miglia nautiche, limitata dalla propagazione VHF e dalle condizioni meteorologiche [BPAFT25], ma può essere estesa tramite reti satellitari Satellite AIS (AIS-sat) [Int26].

2.1.2 Struttura e codifica del messaggio AIS

I messaggi AIS non vengono trasmessi in chiaro come testo, ma sono codificati in formato binario compresso (utilizzando una codifica a 6 bit) per minimizzare l'occupazione della banda radio VHF [Int26]. I diversi tipi di messaggi (da 1 a 27) rispondono a specifiche esigenze; tra questi, i messaggi di Tipo 1, 2 e 3 sono i più frequenti e vengono impiegati dalle navi di Classe A per comunicare periodicamente i dati di posizione dinamica [Int26]. La tabella 2.1 descrive dettagliatamente la ripartizione dei bit all'interno di un frame AIS di Posizione standard.

¹<https://www.marinetraffic.com>

²<https://www.vesselfinder.com>

Bit	Nome Campo	Descrizione
6	Message ID	Identifica il tipo di messaggio (valori: 1, 2 o 3).
2	Repeat Indicator	Numero di ripetizioni del messaggio.
30	Source ID (MMSI)	Codice univoco di 9 cifre della nave.
4	Navigational Status	Stato della nave (es. 0 = in navigazione, 1 = all'ancora, 5 = ormeggiata).
8	ROT	Velocità di accostata (virata) a destra o sinistra.
10	SOG	Velocità rispetto al fondo (risoluzione 0.1 nodi).
1	Position Accuracy	Accuratezza del GPS (1 = alta, ≤ 10 m; 0 = bassa, > 10 m).
28	Longitude	Longitudine in 1/10000 di minuto d'arco.
27	Latitude	Latitudine in 1/10000 di minuto d'arco.
12	COG	Rotta rispetto al fondo (in decimi di grado, 0–3599).
9	True Heading	Prua vera della nave (0–359 gradi).
6	Time Stamp	Secondo della coordinata temporale UTC di generazione (0–59).
2	Special Manoeuvre	Indicatore di manovre speciali.
2	Spare	Bit di riserva inutilizzati (impostati a 0).
1	Transmit Power	Potenza di trasmissione.
1	RAIM flag	Indicatore di integrità del ricevitore GPS.
19	Communication State	Dati di sincronizzazione e gestione degli slot temporali.
168	Totale	Struttura che occupa 1 slot di trasmissione.

Tabella 2.1: Struttura del payload di un messaggio AIS di Posizione (Tipo 1, 2, 3).

2.1.3 Aspetti normativi e applicazioni

L'uso del Sistema di Identificazione Automatica è regolato dall'IMO ed è obbligatorio ai sensi della convenzione Safety Of Life At Sea (SOLAS) per le navi commerciali in viaggio internazionale con stazza lorda pari o superiore a 300 Gross Tonnage (GT), e per tutte le navi passeggeri indipendentemente dalle loro dimensioni [BPW14, Uni15]. Oltre all'identificazione e al monitoraggio dei corridoi marittimi, l'AIS è oggi impiegato in diversi ambiti, quali:

- la prevenzione delle collisioni in tempo reale (Closest Point of Approach (CPA)), che permette ai sistemi di bordo di innescare allarmi visivi e acustici qualora si rilevi una possibile collisione [BPW14];
- le operazioni di ricerca e soccorso (Search and Rescue (SAR)) e la gestione di uomini in mare (Man OverBoard (MOB)): i trasmettitori di soccorso (es. AIS-Search and Rescue Transponder (AIS-SART)) si attivano a contatto con l'acqua trasmettendo la posizione GPS a tutte le unità limitrofe, utilizzando messaggi specifici e un prefisso MMSI dedicato [BPW14, Int26];
- la segnalazione di ostacoli tramite ausili alla navigazione (Aids to Navigation (AtoN)): boe, fari o stazioni possono trasmettere le proprie coordinate o simulare ostacoli virtuali (Virtual AtoN) per deviare il traffico da aree pericolose o fondali bassi [BPW14, Int26];
- il monitoraggio ambientale, ad esempio per rintracciare le navi responsabili di versamenti illegali di petrolio in mare [BPW14].

2.1.4 Limiti e problemi aperti

Nonostante l'ampia diffusione, sono presenti alcuni limiti e vulnerabilità dell'AIS:

- natura non autenticata e vulnerabilità informatica: i dati generati a bordo possono essere: manipolati (*spoofed*) per creare navi inesistenti o falsi allarmi SAR/CPA, alterati sovrascrivendo i segnali (*hijacking*), oppure resi indisponibili tramite attacchi radio *Denial of Service*, come il blocco del salto di frequenza (*frequency hopping*) o l'esaurimento degli slot disponibili (*slot starvation*) [BPW14];

- limiti fisici della trasmissione VHF: il canale offre una larghezza di banda molto stretta (circa 10 kbps) a fronte di una copertura limitata Line of Sight (tipicamente 40 km, fortemente dipendente dalle condizioni meteorologiche) [BPAFT25];
- incompatibilità con i requisiti delle navi a guida autonoma (Maritime Autonomous Surface Ships (MASS)): questi mezzi richiedono sensori avanzati come telecamere ad alta risoluzione, Light Detection and Ranging (LiDAR) e radar, generando flussi di dati continui nell'ordine dei Mbps che le attuali reti marittime non sono in grado di sostenere [BPAFT25].

In particolare, questo lavoro di tesi affronterà l'ultimo problema elencato sopra, utilizzando la programmazione aggregata (sezione 2.2).

2.2 Aggregate Programming, SCR e Kollektive

2.2.1 Programmazione Aggregata (AP)

La programmazione di sistemi distribuiti moderni, come l'Internet of Things (IoT) o le reti navali, risulta complessa a causa dell'eterogeneità dei dispositivi, dei guasti frequenti e della necessità di interazioni basate sulla prossimità fisica [BPV15]. Per far fronte a questi problemi, Aggregate Programming (AP) propone un cambio di paradigma: l'unità base della computazione non è più il singolo dispositivo, ma un intero insieme (o aggregato) di nodi cooperanti, astraendo dettagli come il numero e l'esatta posizione dei singoli apparati [BPV15, BV16]. AP è un approccio di macro-programmazione funzionale basato sul concetto di campo computazionale (computational field), ovvero una struttura dati distribuita che mappa ogni dispositivo in un dato spazio e tempo a un valore [VAB⁺18, BPAFT25]. I dispositivi eseguono tutti un medesimo programma in modo asincrono (in round continui) seguendo un modello a tre fasi [BPAFT25]:

- *sense*: il nodo raccoglie informazioni dall'ambiente locale e dai messaggi ricevuti dai vicini durante la fase di riposo;
- *compute*: il programma aggregato viene valutato usando i dati raccolti, producendo un nuovo stato locale e i messaggi da condividere;

- *share*: il dispositivo invia i messaggi ai propri vicini.

2.2.2 Field Calculus (FC) e Building Blocks

Il fondamento matematico di Aggregate Programming è Field Calculus (FC), un calcolo universale minimale che modella le interazioni dei dispositivi nello spazio e nel tempo [BV16, VAB⁺18]. FC opera a un basso livello di astrazione, basandosi su un numero ridotto di costrutti [VAB⁺18]:

- **nbr** (*Neighborhood values*): permette l'interazione del dispositivo con il vicinato, raccogliendo e mappando i valori più recenti di una determinata espressione provenienti dai nodi vicini e dal nodo stesso [VAB⁺18, BV14];
- **rep** (*Time evolution*): modella la dinamica temporale, inizializzando una variabile di stato locale e aggiornandola ciclicamente a ogni round di computazione [VAB⁺18, BV14];
- **if** (*Domain restriction*): partiziona la rete in sottoregioni in cui vengono eseguiti rami di codice differenti [VAB⁺18].

Per aumentare il livello di astrazione ed evitare la gestione manuale dei costrutti come **rep** e **nbr**, come illustrato in fig. 2.1, sono stati identificati una serie di building blocks formalmente provati per essere auto-stabilizzanti. Questi blocchi incapsulano i costrutti di FC e ciò garantisce che il sistema che li utilizza sia in grado di recuperare da guasti e adattarsi ai cambiamenti topologici, convergendo sempre verso lo stato corretto [BV14, VAB⁺18]. I principali operatori di base sono [BV14, BPV15]:

- **G**: diffonde informazioni da una regione sorgente seguendo un gradiente (es. calcolo delle distanze minime o broadcast);
- **C**: accumula informazioni muovendosi lungo il gradiente di un potenziale (tipicamente verso un nodo pozzo o sink);
- **T**: regola l'evoluzione temporale, agendo come una memoria a tempo o un count-down;

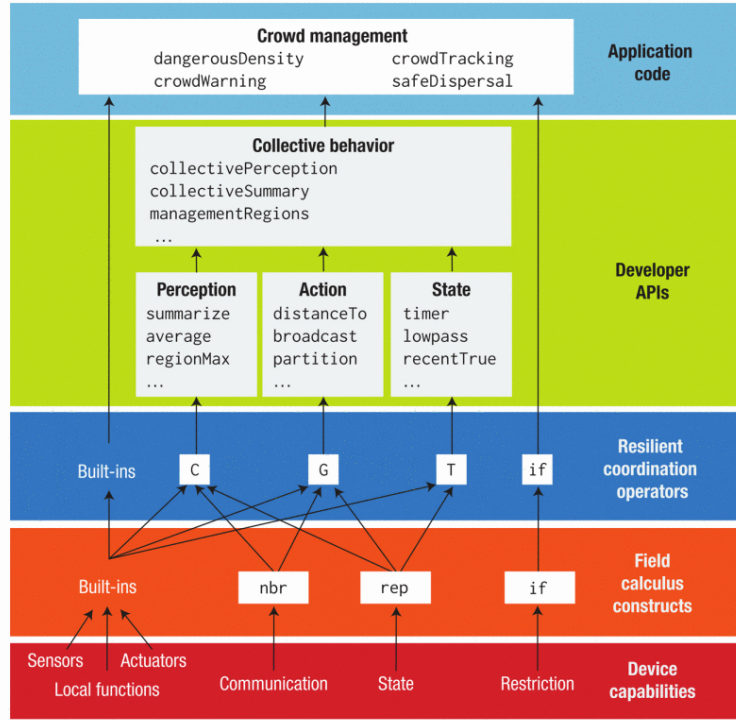


Figura 2.1: Livelli di astrazione della programmazione aggregata. Le capacità software e hardware di alcuni dispositivi sono utilizzate per implementare costrutti di FC a livello aggregato. Questi costrutti sono utilizzati per implementare operazioni di coordinamento a blocchi resilienti, che vengono poi racchiuse e combinate insieme per produrre API. © 2015 IEEE. Ristampata, con permesso, da [BPV15].

- **S**: elegge dei leader locali in modo sparso, frammentando lo spazio in parti-
zioni (es. diagrammi di Voronoi).

2.2.3 Self-organising Coordination Regions (SCR)

Sfruttando i building blocks, è stato ideato un design pattern chiamato Self-organising Coordination Regions (SCR), pensato appositamente per gestire reti molto grandi e complesse, come quelle dell'Edge Computing [CPVN19]. Il suo obiettivo è trovare un compromesso efficace tra un controllo totalmente centralizzato e uno completamente decentralizzato [CPVN19]. Il sistema si auto-organizza suddividendosi spontaneamente in regioni. All'interno di ciascuna regione, le attività vengono coordinate in modo autonomo grazie a un continuo scambio di informazioni (un ciclo di feedback, come mostrato in fig. 2.2) tra un nodo responsabile (leader) e i dispositivi circostanti [CPVN19]. Il funzionamento di questo pattern si basa su quattro processi continui [CPVN19]:

1. *Elezione dei leader*: vengono eletti i leader da un insieme di candidati (tramite l'operatore S).
2. *Formazione delle regioni*: ogni nodo viene assegnato a un singolo leader, costruendo i percorsi di comunicazione ottimali.
3. *Raccolta delle informazioni*: flussi di dati si muovono dai nodi della regione verso il leader, utilizzando algoritmi come C.
4. *Propagazione delle informazioni*: i leader valutano i dati raccolti e diffondono i risultati o le direttive verso tutti i membri della propria regione (utilizzando G).

Il pattern SCR è resiliente: se un leader subisce un guasto o si verificano variazioni nel carico, il sistema riconfigura automaticamente i leader e ridimensiona le regioni in pochi secondi [CPVN19].

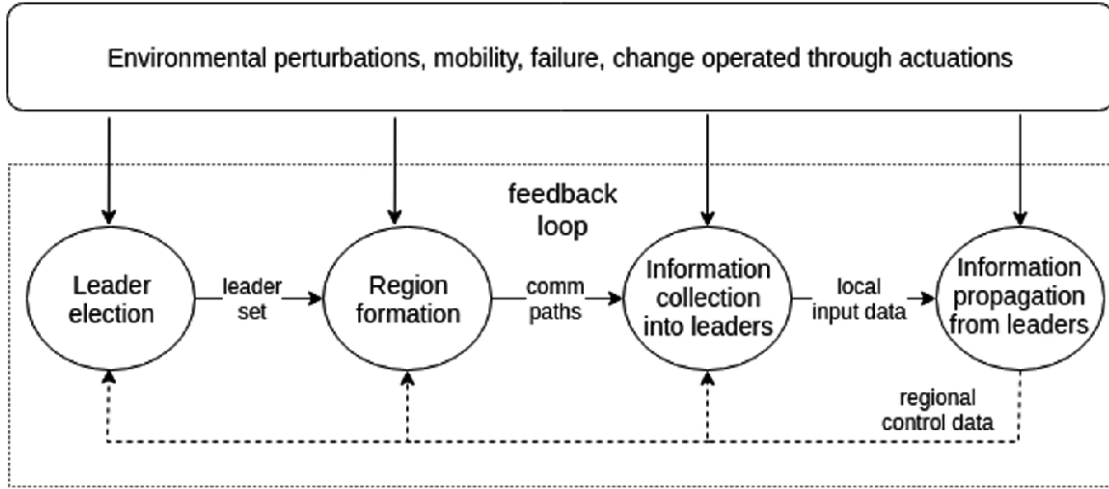


Figura 2.2: Prospettiva dinamica di SCR [CPVN19].

2.2.4 Kollektive

Dal punto di vista dell'implementazione, la programmazione aggregata viene concretizzata tramite Domain Specific Languages (DSL), tra cui ScaFi³ (basato su Scala) e Protelis⁴ (basato su Java). In questa tesi viene utilizzato **Kollektive**⁵, un DSL nativo basato su Kotlin. È un framework open-source progettato per semplificare lo sviluppo di sistemi distribuiti attraverso l'utilizzo dei principi di AP [Col26]. Architetturealmente, Kollektive è suddiviso in tre macro-componenti:

1. un plugin per il compilatore, che si occupa di gestire l'allineamento;
2. il DSL, che definisce i costrutti linguistici fondamentali;
3. e la libreria standard, che fornisce le funzioni essenziali per AP.

Il DSL Kollektive espone funzioni per la gestione dello stato e delle interazioni di rete, ad esempio [BPAFT25]:

- **neighboring**: consente di osservare e mappare in un campo i valori di una specifica variabile nei nodi vicini. Corrisponde a `nbr` in FC;

³<https://scafi.github.io/>

⁴<https://protelis.github.io/wiki-beta/>

⁵<https://kollektive.github.io>

- **exchange**: è uno degli operatori più importanti del DSL. Modella la comunicazione *anisotropica*, consentendo a un nodo di inviare messaggi differenti a vicini diversi.
- **share**: modella la comunicazione *isotropica* (invia lo stesso stato a tutti), combinando lo stato precedente con le informazioni scambiate nel vicinato per evolvere i dati nello spazio e nel tempo;
- **evolve**: equivale a **rep** nel FC e consente di modellare l'evoluzione dello stato del dispositivo nel tempo;
- **alignedOn**: introduce la segmentazione del dominio (domain segmentation), isolando le comunicazioni ed eseguendo i blocchi di codice esclusivamente tra i dispositivi che soddisfano e condividono un medesimo identificatore (pivot) [BPAFT25].

I building blocks vengono implementati componendo queste primitive. Operazioni come **gradientCast** o il calcolo delle distanze tramite Bellman-Ford si ottengono combinando **exchange** o **share** con una metrica, ad esempio calcolando la formula di Haversine sui dati GPS recuperati tramite **neighboring** [BPAFT25].

Per quanto concerne l'implementazione del pattern SCR, Collektive vi provvede utilizzando algoritmi di elezione e segmentazione. La formazione dinamica delle regioni è delegata alla funzione **boundedElection**, che partiziona la rete senza alcuna coordinazione centrale [BPAFT25]. L'algoritmo riceve in input il diametro massimo del cluster e una metrica di “forza” del nodo (la priorità del candidato). Questa “forza” coincide con l'eleggibilità del dispositivo a diventare leader (che può basarsi su risorse energetiche, di calcolo, o sul data rate come nel caso dell'esperimento iniziale di questa tesi [BPAFT25]). È possibile personalizzare l'elezione del leader utilizzando **Candidacy** (rappresenta la candidatura in un'elezione di un candidato) all'interno del parametro **selectBest** in **boundedElection**. Valutando le “forze”, **boundedElection** risolve le contese ed elegge i leader, adattandosi tempestivamente ai guasti o ai cambiamenti topologici [BPAFT25]. Una volta eletti i leader, all'interno della regione viene utilizzato l'operatore **alignedOn(leader)**: in questo modo i dispositivi di una stessa regione restringono il proprio dominio alla sola regione di appartenenza, instaurando canali diretti (come dei **convergeCast**)

Listing 2.1: Esempio di codice Kollektive per una semplice simulazione di programmazione aggregata.

```
1 fun Aggregate<Int>.temperatureEntrypoint(env: EnvironmentVariables,
2   kollektiveDevice: KollektiveDevice<*>): Boolean {
3   val temp: Double = env["temperature"]
4   val distances = with(kollektiveDevice) { distances() }
5   val isLeader = (boundedElection(bound = 20.0, metric = distances) == localId)
6   .also { env["leader"] = it }
7   val closestLeader = gradientCast(source = isLeader, local = localId, metric =
8     distances)
9   return alignedOn(closestLeader) {
10    val regDistances = with(kollektiveDevice) { distances() }
11    val count = countDevices(sink = isLeader)
12    val sum = convergeSum(sink = isLeader, local = temp)
13    val avg = if (isLeader && count > 0) sum / count else 0.0
14    val alarmDecision = isLeader && avg > 30.0
15    gradientCast(source = isLeader, local = alarmDecision, metric =
16      regDistances)
17    .also { env["alarm"] = it }
18  }
19 }
```

per instradare le informazioni senza interferire con le regioni adiacenti [BPAFT25]. `temperatureEntrypoint` (listing 2.1) rappresenta un esempio di un semplice algoritmo in cui vengono utilizzate le nozioni di SCR e AP: i nodi possiedono un attributo “temperatura”. Essi si organizzano in regioni con il proprio leader e quest’ultimo si occupa di calcolare la media delle temperature dei dispositivi della propria area. Se la media supera i 30°C allora il leader diffonde un allarme.

2.3 Simulazione in ambito marittimo

2.3.1 Modeling & Simulation (M&S)

I porti marini e gli scenari di navigazione costiera sono sistemi altamente complessi, caratterizzati da un'elevata densità di traffico, condizioni meteo variabili e diversi attori (navi mercantili, piccoli natanti, rimorchiatori, e torri di controllo). In tale contesto, testare nuove configurazioni di rete, valutare l'efficienza delle operazioni o formare il personale sul campo risulta spesso proibitivo per questioni di sicurezza, limiti fisici ed elevati costi operativi [CPL15].

Per risolvere queste problematiche, il paradigma Modeling & Simulation (M&S) rappresenta lo strumento d'eccellenza. M&S consente di riprodurre il comportamento del sistema reale attraverso un modello computazionale, offrendo un ambiente virtuale in cui analizzare scenari “what-if”, condurre analisi sugli incidenti e massimizzare l'efficacia del training [CPL15]. In particolare, la simulazione distribuita permette l'addestramento congiunto di piloti di navi e operatori del traffico (VTS) all'interno di un unico ambiente virtuale condiviso, riducendo i rischi e i costi associati agli errori in fase di apprendimento [CPL15]. Oltre alla formazione del personale, la simulazione si rivela oggi un mezzo fondamentale per lo sviluppo e la prototipazione di reti di telecomunicazione e schemi di coordinamento adattivo destinati alle future navi a guida autonoma (MASS) [BPAFT25, CPL15].

2.3.2 Alchemist

Per l'esecuzione e la valutazione dell'esperimento di questa tesi è stato adottato **Alchemist**⁶, una piattaforma di simulazione di sistemi distribuiti, particolarmente adatta alla programmazione aggregata. Consente di modellare e simulare le interazioni tra agenti in ambienti complessi, supportando diverse astrazioni e paradigmi di programmazione [Col26].

⁶<https://alchemistsimulator.github.io/>

Meta-modello e Astrazioni

Per garantire la flessibilità necessaria a simulare scenari complessi, la logica di Alchemist, come mostrato in fig. 2.3, si basa su un meta-modello le cui entità fondamentali sono [Pia21]:

- *Environment*: lo spazio in cui vivono i nodi, con la responsabilità di mapparne la posizione. Può includere ostacoli fisici, mappe reali (es. file OpenStreetMap⁷ o tracce GPS) e ambienti indoor [Pia21].
- *Node*: un contenitore situato nell'ambiente che ospita reazioni e "molecole".
- *Linking rule*: una funzione dell'ambiente che associa ogni nodo a un vicinato.
- *Molecule* e *Concentration*: la molecola rappresenta un dato, mentre la concentrazione ne rappresenta il valore associato all'interno di uno specifico nodo.
- *Reaction*: un evento che modifica lo stato dell'ambiente. È composto da condizioni (che ne determinano l'eseguibilità), una distribuzione temporale, un'equazione di tasso (*rate*) e azioni concrete (es. movimento del nodo o alterazione di una concentrazione) [PMV13, Pia21].

⁷<https://www.openstreetmap.org/>

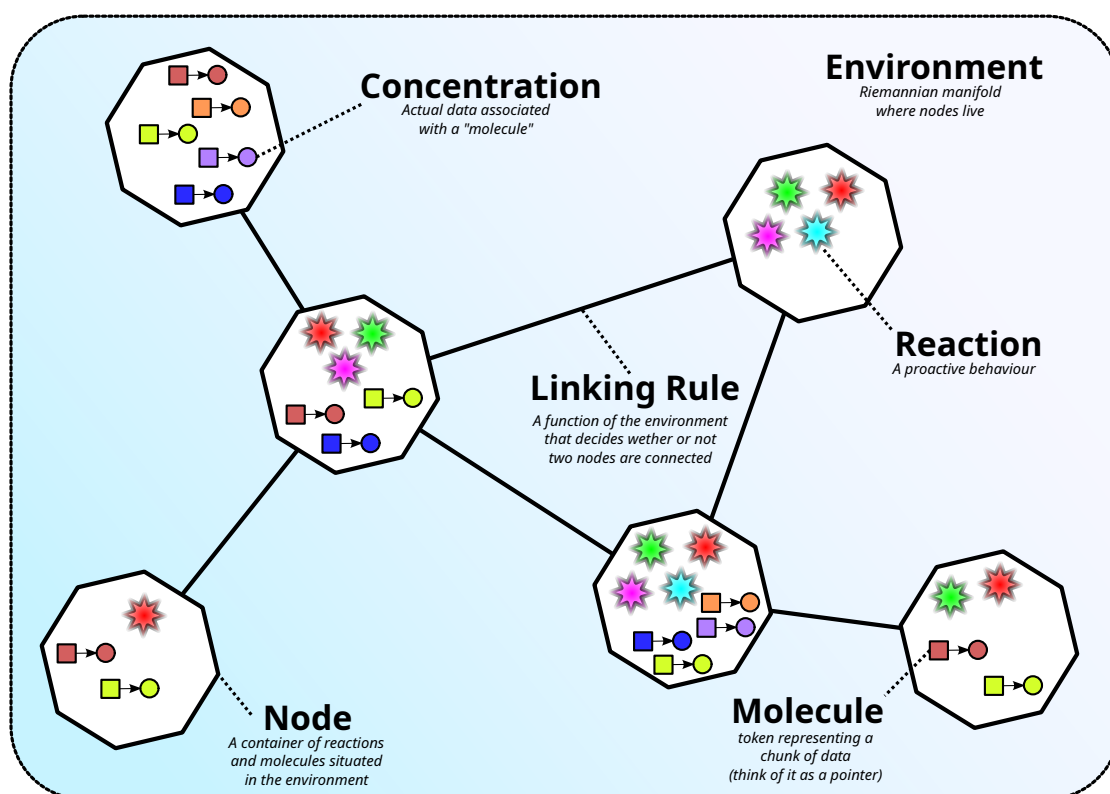


Figura 2.3: Rappresentazione grafica del meta-modello di Alchemist [Pia21].

Incarnazioni (*Incarnations*) per Aggregate Programming

Per supportare linguaggi e paradigmi differenti, Alchemist sfrutta il concetto di *Incarnation* (incarnazione), che permette di definire il tipo di dato esatto usato per le variabili (Concentrazione) e di fornire comportamenti specifici [Pia21]. Tra le incarnazioni supportate, assumono particolare rilevanza: `collektive`, `protelis` e `scafi`. Queste incarnazioni sono progettate specificamente per eseguire programmi basati su AP [Pia21]. Insieme, Alchemist e Collektive, consentono di implementare programmi aggregati in Kotlin che possono essere eseguiti e simulati in ambienti dinamici e configurabili [Col26].

Configurazione e Riproducibilità

Alchemist semplifica la configurazione degli ambienti di test affidandosi a specifiche scritte in *YAML*, un linguaggio di serializzazione dati in formato dichiarativo [Pia21]. Tramite un file *YAML* è possibile definire per esempio i modelli di rete (*network-model*) e la disposizione iniziale dei nodi (*deployments*). In listing 2.2 viene mostrata la struttura di una possibile configurazione di una simulazione, riprendendo l'esempio: listing 2.1. Un aspetto pratico fondamentale di Alchemist per le simulazioni in ambito marittimo è il supporto nativo a dati geospaziali. Il simulatore permette di importare ambienti fisici basati su OpenStreetMap e di impostare la dinamica dei nodi importando direttamente tracce GPS in formato GPS eXchange Format (GPX) (generate, ad esempio, dalla decodifica di messaggi AIS), consentendo di testare algoritmi sulle rotte effettivamente percorse dalle imbarcazioni [Pia21, BPAFT25]. Infine, Alchemist garantisce la ripetibilità degli esperimenti: il sistema separa e controlla in modo deterministico i seed (semi) dei generatori pseudo-casuali (come il Mersenne Twister) sia per la disposizione iniziale dei nodi, sia per l'esecuzione della simulazione [Pia21].

Listing 2.2: Parte della configurazione della simulazione per il programma: listing 2.1.

```
1 incarnation: collective
2
3 network-model:
4   type: ConnectWithinDistance
5   parameters: [3.0]
6
7 _pool: &program
8   - time-distribution: 1
9     type: Event
10    actions:
11      - type: RunCollectiveProgram
12        parameters: [it.unibo.collective.TemperatureKt.temperatureEntrypoint]
13
14 deployments:
15   - type: Grid
16     parameters: [ 0, 0, 40, 80, 2, 2, 0.5, 0.5 ]
17     programs:
18       - *program
19     contents:
20       - molecule: leader
21         concentration: false
22       - molecule: temperature
23         concentration: 17.0
```

2.4 Algoritmo precedente

L'esperimento su cui si basa questa tesi ha come obiettivo quello di migliorare la raccolta e la trasmissione di dati in uno scenario marittimo realistico. Il caso di studio considera il fiordo di Kiel, in Germania, dove le navi sono modellate come nodi in grado di comunicare tra loro e con stazioni a terra [BPAFT25]. Il problema affrontato nasce dal divario tra la quantità di dati generata dai moderni sistemi di navigazione autonoma (MASS) e la capacità delle infrastrutture di comunicazione marittima tradizionali. Navi autonome o semi-autonome possono infatti produrre flussi di dati ad alta intensità, derivanti ad esempio da telecamere e LiDAR; al contrario, tecnologie tradizionali come AIS e sistemi basati su VHF offrono una capacità trasmissiva limitata [BPAFT25]. Nella simulazione, ogni imbarcazione è dotata di diversi dispositivi di comunicazione: un sistema Automatic Packet Reporting System (APRS) su VHF, un dispositivo Long Range Wide Area Network (LoRaWAN) di classe C, un modulo 5G di tipo consumer e un dispositivo compatibile con Wi-Fi Direct [BPAFT25]. Inoltre, alcune imbarcazioni possono essere equipaggiate con una torre 5G montata a bordo, con probabilità indicata

dal parametro p_{5G} . Ogni nave tenta di trasmettere verso terra dei dati di dimensione fissata pari a $d = 3$ Mbit/s, valore scelto come approssimazione del bitrate necessario per un flusso video compresso a 720p [BPAFT25].

Per stimare la qualità dei collegamenti, l'esperimento associa a ogni coppia di nodi un data rate stimato in funzione della tecnologia disponibile e della distanza tra i dispositivi. Le tecnologie considerate sono quindi 5G, Wi-Fi, LoRaWAN e APRS. A partire da valori di riferimento ricavati dalla letteratura, il data rate in linea di vista viene stimato mediante interpolazione log-lineare rispetto alla distanza [BPAFT25].

Gli approcci confrontati nell'esperimento precedente sono quattro:

- *Baseline non cooperativa*: rappresenta lo stato dell'arte di riferimento. Ogni nave tenta di comunicare direttamente con una stazione a terra utilizzando la tecnologia più favorevole tra quelle disponibili. Se il collegamento diretto offre un data rate sufficiente a trasmettere il flusso d , la comunicazione viene considerata riuscita; in caso contrario, il flusso viene scartato [BPAFT25]. Questo approccio non prevede alcuna collaborazione tra imbarcazioni.
- *Distance-based Multi-Relay Communication (Dist-MR)*: ogni nave che non riesce a raggiungere direttamente la stazione a terra seleziona come relay un'imbarcazione vicina lungo il cammino geograficamente più breve verso la destinazione. Si ha quindi un insieme di alberi che originano nelle stazioni a terra. La metrica usata per la scelta del relay è la distanza geografica [BPAFT25].
- *Data Rate-based Multi-Relay Communication (DR-MR)*: modifica il criterio di scelta del relay utilizzato in Dist-MR. Invece di minimizzare la distanza geografica, l'obiettivo è selezionare il percorso che massimizza il data rate disponibile verso la stazione a terra [BPAFT25].
- *Collective Summarisation Clusters (CSC)*: organizza le imbarcazioni in cluster auto-organizzanti. Ogni cluster possiede un leader, responsabile della raccolta dei flussi dati prodotti dai membri della regione e della loro successiva sintesi prima dell'invio verso terra o verso un altro cluster. Questo approccio si basa sul pattern SCR: i leader vengono eletti dinamicamente e

la formazione dei cluster tiene conto della capacità di trasmettere dati verso il leader [BPAFT25].

Capitolo 3

Analisi

3.1 Limiti dell'algoritmo esistente

3.2 Obiettivi

3.3 Requisiti

Capitolo 4

Design e implementazione

- 4.1 Strategia di integrazione nel criterio di clustering
- 4.2 Estrazione e integrazione delle feature AIS aggiuntive
- 4.3 Modifiche all'algoritmo

Capitolo 5

Valutazioni

5.1 Risultati dell'esperimento precedente

5.2 Risultati del nuovo algoritmo

5.3 Confronto e discussione

Capitolo 6

Conclusioni e sviluppi futuri

6.1 Limitazioni dello studio

Elenco delle figure

2.1	Livelli di astrazione della programmazione aggregata. Le capacità software e hardware di alcuni dispositivi sono utilizzate per implementare costrutti di FC a livello aggregato. Questi costrutti sono utilizzati per implementare operazioni di coordinamento a blocchi resilienti, che vengono poi racchiuse e combinate insieme per produrre API. © 2015 IEEE. Ristampata, con permesso, da [BPV15].	9
2.2	Prospettiva dinamica di SCR [CPVN19].	11
2.3	Rappresentazione grafica del meta-modello di Alchemist [Pia21]. . .	16

List of Listings

2.1	Esempio di codice Collettive per una semplice simulazione di programmazione aggregata.	13
2.2	Parte della configurazione della simulazione per il programma: listing 2.1.	18

Bibliografia

- [BPAFT25] Martina Baiardi, Danilo Pianini, Ghassan Al-Falouji, and Sven Tomforde. Robust communication through collective adaptive relay schemes for maritime vessels. In *2025 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, pages 21–32, 2025.
- [BPV15] Jacob Beal, Danilo Pianini, and Mirko Viroli. Aggregate programming for the internet of things. *Computer*, 48(9):22–30, 2015.
- [BPW14] Marco Balduzzi, Alessandro Pasta, and Kyle Wilhoit. A security evaluation of ais automated identification system. In *Proceedings of the 30th Annual Computer Security Applications Conference, ACSAC '14*, page 436–445, New York, NY, USA, 2014. Association for Computing Machinery.
- [BV14] Jacob Beal and Mirko Viroli. Building blocks for aggregate programming of self-organising applications. In *2014 IEEE Eighth International Conference on Self-Adaptive and Self-Organizing Systems Workshops*, pages 8–13, 2014.
- [BV16] Jacob Beal and Mirko Viroli. Aggregate programming: From foundations to applications. In *Formal Methods for the Quantitative Evaluation of Collective Adaptive Systems*, pages 233–260. Springer, 2016.

- [Col26] Kollektive Organization. Kollektive: Practical. scalable. kotlin-first aggregate programming. <https://kollektive.github.io/>, 2026. Accessed: 2026.
- [CPL15] Alessandro Chiurco, Leonardo Pagnotta, and Francesco Longo. *Ship bridge simulators for training in maritime domain*. PhD thesis, Scuola di Dottorato Pitagora, 2015.
- [CPVN19] Roberto Casadei, Danilo Pianini, Mirko Viroli, and Antonio Natali. Self-organising coordination regions: A pattern for edge computing. In *Coordination Models and Languages: 21st IFIP WG 6.1 International Conference, COORDINATION 2019, Held as Part of the 14th International Federated Conference on Distributed Computing Techniques, DisCoTec 2019, Kongens Lyngby, Denmark, June 17–21, 2019, Proceedings*, page 182–199, Berlin, Heidelberg, 2019. Springer-Verlag.
- [Int26] International Telecommunication Union (ITU). Technical characteristics for an automatic identification system using time division multiple access in the maritime mobile service. Recommendation Recommendation ITU-R M.1371-6, ITU Radiocommunication Sector, Geneva, Switzerland, 02 2026.
- [Pia21] Danilo Pianini. Simulation of Large Scale Computational Ecosystems with Alchemist: A Tutorial. In Miguel Matos and Fabíola Greve, editors, *Lecture Notes in Computer Science*, volume LNCS-12718 of *Distributed Applications and Interoperable Systems*, pages 145–161, Valletta, Malta, June 2021. Springer International Publishing. Part 5: Invited Paper.
- [PMV13] D Pianini, S Montagna, and M Viroli. Chemical-oriented simulation of computational systems with alchemist. *Journal of Simulation*, 7(3):202–215, August 2013.
- [Uni15] United States Coast Guard. 2015 code of federal regulations implementing the ais provisions of the safety of life at sea (solas) convention

and marine transportation security act of 2002, 2015. Includes IMO Resolution MSC.99(73) and U.S. Code Title 46.

- [VAB⁺18] Mirko Viroli, Giorgio Audrito, Jacob Beal, Ferruccio Damiani, and Danilo Pianini. Engineering resilient collective adaptive systems by self-stabilisation. *ACM Trans. Model. Comput. Simul.*, 28(2), March 2018.

Acknowledgements

Optional. Max 1 page.